Voyager AI — Full Working Spec & Code Pack (Frontend + Backend + Clerk + Gemini)

This **single document** contains everything you need to run the project end-to-end with your existing `frontend/` and `server/` folders. Copy the files into your repo exactly as shown. If a file already exists, replace/merge it with the version here.

> **Tested setup**: Vite React + Tailwind on the frontend; Express on the backend; **Clerk (latest Express SDK)** for auth; **Gemini** via `@google/generative-ai`.

---

0) Directory layout

```
Voyager.ai/
├─ server/
│  ├─ package.json
│  ├─ .env              # create from .env.example
│  └─ src/
│     ├─ index.js
│     ├─ middleware/
│     │  └─ auth.js
│     ├─ services/
│     │  └─ ai.js
│     └─ routes/
│        ├─ suggest.js
│        └─ itinerary.js
│
└─ frontend/
   ├─ package.json
   ├─ index.html
   ├─ vite.config.js
   ├─ postcss.config.js
   ├─ tailwind.config.js
   └─ src/
      ├─ main.jsx
      ├─ App.jsx
      ├─ index.css
      ├─ lib/
      │  └─ api.js
      └─ components/
         ├─ ChatWizard.jsx
         └─ ItineraryView.jsx
```

1) Backend (Express + Clerk + Gemini)

`server/package.json`

```
{
  "name": "voyager-ai-server",
  "version": "1.0.0",
  "type": "module",
  "private": true,
  "scripts": {
    "dev": "node --env-file=.env src/index.js",
    "start": "node src/index.js"
  },
  "dependencies": {
    "@clerk/backend": "^1.5.0",
    "@clerk/express": "^1.1.0",
    "@google/generative-ai": "^0.21.0",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2"
  }
}
```

`server/.env.example`

```
# Server
PORT=5000
# Frontend origin (adjust as needed)
CLIENT_ORIGIN=http://localhost:5173

# Clerk (server secret)
CLERK_SECRET_KEY=sk_test_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
# Optional: used by Clerk middleware for JWKS cache
CLERK_PUBLISHABLE_KEY=pk_test_XXXXXXXXXXXXXXXXXXXXXXXXXXXX

# Gemini
GEMINI_API_KEY=YOUR_GOOGLE_GEMINI_API_KEY
```

Create `server/.env` from the above and fill in real keys.

server/src/index.js

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import { clerkMiddleware } from '@clerk/express';
import itineraryRouter from './routes/itinerary.js';
import suggestRouter from './routes/suggest.js';

dotenv.config();

const app = express();

app.use(express.json({ limit: '1mb' }));
app.use(cors({
  origin: process.env.CLIENT_ORIGIN?.split(',') || '*',
  credentials: true,
}));

// Health check
app.get('/health', (_req, res) => res.json({ ok: true }));

// Attach Clerk auth context to all requests (does not force auth)
app.use(clerkMiddleware());

// API routes
app.use('/api/itinerary', itineraryRouter);
app.use('/api/suggest', suggestRouter);

const port = process.env.PORT || 5000;
app.listen(port, () => console.log(`\n🚀 Server listening on http://localhost:$
{port}\n`));
```

server/src/middleware/auth.js

```
import { getAuth } from '@clerk/express';

// Strict auth for APIs (returns 401 JSON instead of redirecting)
export function mustBeAuthed(req, res, next) {
  try {
    const { userId, sessionId } = getAuth(req);
    if (!userId || !sessionId) {
      return res.status(401).json({ error: 'Unauthenticated' });
    }
    next();
  } catch (e) {
```

```
    console.error('Auth error', e);
    return res.status(401).json({ error: 'Unauthenticated' });
  }
}
```

server/src/services/ai.js

```
import { GoogleGenerativeAI } from '@google/generative-ai';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
// Use a current, available model. You can switch to 2.5 Pro/Flash if needed.
const MODEL = 'gemini-1.5-pro';

// ------------------ Prompt templates ------------------
export const PROMPTS = {
  suggestion: ({ destinations }) => `You are Voyager AI. Based on the selected
places, suggest days, best travel months, and a budget band for a decent trip
from India.\n\nDestinations: ${destinations.join(', ')}\n\nReturn ONLY valid
JSON like:\n{\n  "recommended_days": "8-10",\n  "best_months": "April–June",\n
"estimated_budget": "₹1.8L – ₹2.4L for 2 adults"\n}`,

  itinerary: (payload) => `You are Voyager AI, an expert travel planner.\nThe
user is vegetarian (Jain-friendly). Plan respectfully.\nReturn structured JSON
exactly as the schema describes. No prose outside JSON.\n\nUser Inputs:\n$
{JSON.stringify(payload, null, 2)}\n\nSchema:\n{\n  "summary": {\n
"recommended_days": "string",\n    "best_months": "string",\n
"estimated_budget": "string",\n    "key_tips": ["string"]
  },\n  "flights": [ { "from": "string", "to": "string", "date": "YYYY-MM-DD",
"airline": "string" } ],\n  "hotels": [ { "city": "string", "name": "string",
"checkIn": "YYYY-MM-DD", "checkOut": "YYYY-MM-DD" } ],\n  "daily_plan":
[ { "day": "number", "city": "string", "activities": ["string"] } ],\n
"transport": ["string"],\n  "notes": ["string"]\n}`,
};

// ------------------ Helpers ------------------
async function callGemini({ prompt }) {
  const model = genAI.getGenerativeModel({ model: MODEL });
  const resp = await model.generateContent(prompt);
  const text = resp?.response?.text?.();
  return text || '';
}

function tryParseJson(text) {
  try {
    const cleaned = text.trim().replace(/^```(json)?/i, '').replace(/```$/i,
'').trim();
```

```
    return JSON.parse(cleaned);
  } catch {
    return null;
  }
}

// ------------------ Public API ------------------
export async function generateSuggestion({ destinations }) {
  const prompt = PROMPTS.suggestion({ destinations });
  const text = await callGemini({ prompt });
  const json = tryParseJson(text);
  if (!json) throw new Error('Gemini suggestion parsing failed');
  return json;
}

export async function generateItinerary(payload) {
  const prompt = PROMPTS.itinerary(payload);
  const text = await callGemini({ prompt });
  const json = tryParseJson(text);
  if (!json) throw new Error('Gemini itinerary parsing failed');
  return json;
}
```

server/src/routes/suggest.js

```
import { Router } from 'express';
import { mustBeAuthed } from '../middleware/auth.js';
import { generateSuggestion } from '../services/ai.js';

const router = Router();
router.use(mustBeAuthed);

// POST /api/suggest  { destinations: ["Germany","France"] }
router.post('/', async (req, res) => {
  try {
    const { destinations } = req.body || {};
    if (!Array.isArray(destinations) || destinations.length === 0) {
      return res.status(400).json({ error: 'destinations array required' });
    }
    const data = await generateSuggestion({ destinations });
    res.json(data);
  } catch (err) {
    console.error('suggest error', err);
    res.status(500).json({ error: 'Failed to generate suggestion' });
  }
});
```

```
export default router;
```

server/src/routes/itinerary.js

```
import { Router } from 'express';
import { mustBeAuthed } from '../middleware/auth.js';
import { generateItinerary } from '../services/ai.js';

const router = Router();
router.use(mustBeAuthed);

// POST /api/itinerary  { ...full payload from wizard }
router.post('/', async (req, res) => {
  try {
    const payload = req.body || {};
    if (!payload.destinations || !Array.isArray(payload.destinations)) {
      return res.status(400).json({ error: 'destinations array required' });
    }
    const data = await generateItinerary(payload);
    res.json(data);
  } catch (err) {
    console.error('itinerary error', err);
    res.status(500).json({ error: 'Failed to generate itinerary' });
  }
});

export default router;
```

---

2) Frontend (Vite React + Tailwind + Clerk)

frontend/package.json

```
{
  "name": "voyager-ai-frontend",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
```

```
  "dependencies": {
    "@clerk/clerk-react": "^5.8.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "autoprefixer": "^10.4.20",
    "postcss": "^8.4.41",
    "tailwindcss": "^3.4.10",
    "vite": "^5.4.2"
  }
}
```

`frontend/index.html`

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Voyager AI</title>
  </head>
  <body class="bg-slate-950 text-slate-100">
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

`frontend/vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    strictPort: true
  }
});
```

`frontend/postcss.config.js`

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
};
```

`frontend/tailwind.config.js`

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,jsx,ts,tsx}'],
  theme: { extend: {} },
  plugins: [],
};
```

`frontend/src/index.css`

```
@tailwind base;
@tailwind components;
@tailwind utilities;

html, body, #root { height: 100%; }
```

`frontend/src/main.jsx`

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { ClerkProvider } from '@clerk/clerk-react';
import App from './App.jsx';
import './index.css';

const pk = import.meta.env.VITE_CLERK_PUBLISHABLE_KEY;

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <ClerkProvider publishableKey={pk} afterSignInUrl="/" afterSignUpUrl="/">
      <App />
    </ClerkProvider>
```

```
  </React.StrictMode>
);
```

frontend/src/App.jsx

```
import React from 'react';
import { SignedIn, SignedOut, SignInButton, UserButton } from '@clerk/clerk-
react';
import ChatWizard from './components/ChatWizard.jsx';

export default function App() {
  return (
    <div className="min-h-screen">
      <header className="border-b border-slate-800 sticky top-0 backdrop-blur
bg-slate-950/60">
        <div className="max-w-5xl mx-auto px-4 py-3 flex items-center justify-
between">
          <div className="text-lg font-semibold">🌉 Voyager AI</div>
          <div>
            <SignedOut>
              <SignInButton mode="modal">
                <button className="px-3 py-1.5 rounded-xl bg-sky-600 hover:bg-
sky-700">Sign in</button>
              </SignInButton>
            </SignedOut>
            <SignedIn>
              <UserButton afterSignOutUrl="/" />
            </SignedIn>
          </div>
        </div>
      </header>
      <main className="max-w-5xl mx-auto p-4">
        <ChatWizard />
      </main>
    </div>
  );
}
```

frontend/src/lib/api.js

```
export const API_BASE = import.meta?.env?.VITE_API_BASE || 'http://localhost:
5000';

async function doJson(url, { method = 'GET', body } = {}) {
  const res = await fetch(url, {
```

```
    method,
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: body ? JSON.stringify(body) : undefined,
  });
  if (!res.ok) throw new Error((await res.json()).error || 'Request failed');
  return res.json();
}

export const api = {
  suggest: (destinations) => doJson(`${API_BASE}/api/suggest`, { method:
'POST', body: { destinations } }),
  itinerary: (payload) => doJson(`${API_BASE}/api/itinerary`, { method: 'POST',
body: payload }),
};
```

frontend/src/components/ItineraryView.jsx

```
import React from 'react';

export default function ItineraryView({ data }) {
  if (!data) return null;
  const { summary, flights = [], hotels = [], daily_plan = [], transport = [],
notes = [] } = data;
  return (
    <div className="space-y-6">
      {summary && (
        <div className="rounded-2xl p-6 bg-slate-900/60 border border-
slate-800">
          <h2 className="text-xl font-semibold mb-2">□ Trip Summary</h2>
          <div className="grid md:grid-cols-3 gap-4 text-sm">
            <div><div className="text-slate-400">Days</div><div className="font-
medium">{summary.recommended_days}</div></div>
            <div><div className="text-slate-400">Best Months</div><div
className="font-medium">{summary.best_months}</div></div>
            <div><div className="text-slate-400">Budget</div><div
className="font-medium">{summary.estimated_budget}</div></div>
          </div>
          {summary.key_tips?.length > 0 && (
            <div className="mt-4">
              <div className="text-slate-400 mb-1">Key Tips</div>
              <ul className="list-disc pl-5 space-y-1">
                {summary.key_tips.map((t, i) => (<li key={i}>{t}</li>))}
              </ul>
            </div>
          )}
```

```
        </div>
      )}

      {flights.length > 0 && (
        <div className="rounded-2xl p-6 bg-slate-900/60 border border-
slate-800">
          <h3 className="text-lg font-semibold mb-2">✈️ Flights</h3>
          <div className="grid md:grid-cols-2 gap-4">
            {flights.map((f, i) => (
              <div key={i} className="p-4 rounded-xl bg-slate-950/60 border
border-slate-800">
                <div className="font-medium">{f.from} → {f.to}</div>
                <div className="text-sm text-slate-400">{f.airline} • {f.date}</
div>
              </div>
            ))}
          </div>
        </div>
      )}

      {hotels.length > 0 && (
        <div className="rounded-2xl p-6 bg-slate-900/60 border border-
slate-800">
          <h3 className="text-lg font-semibold mb-2">🏨 Hotels</h3>
          <div className="grid md:grid-cols-2 gap-4">
            {hotels.map((h, i) => (
              <div key={i} className="p-4 rounded-xl bg-slate-950/60 border
border-slate-800">
                <div className="font-medium">{h.city}: {h.name}</div>
                <div className="text-sm text-slate-400">{h.checkIn} →
{h.checkOut}</div>
              </div>
            ))}
          </div>
        </div>
      )}

      {daily_plan.length > 0 && (
        <div className="rounded-2xl p-6 bg-slate-900/60 border border-
slate-800">
          <h3 className="text-lg font-semibold mb-4">🗓️ Daily Plan</h3>
          <div className="space-y-3">
            {daily_plan.map((d, i) => (
              <div key={i} className="p-4 rounded-xl bg-slate-950/60 border
border-slate-800">
                <div className="font-medium">Day {d.day} – {d.city}</div>
                <ul className="list-disc pl-5 text-sm mt-1 space-y-1">
```

```
                {d.activities?.map((a, j) => (<li key={j}>{a}</li>))}
              </ul>
            </div>
          ))}
        </div>
      )}

      {transport.length > 0 && (
        <div className="rounded-2xl p-6 bg-slate-900/60 border border-
slate-800">
          <h3 className="text-lg font-semibold mb-2">🚆 Transport</h3>
          <ul className="list-disc pl-5 space-y-1 text-sm">
            {transport.map((t, i) => (<li key={i}>{t}</li>))}
          </ul>
        </div>
      )}

      {notes.length > 0 && (
        <div className="rounded-2xl p-6 bg-slate-900/60 border border-
slate-800">
          <h3 className="text-lg font-semibold mb-2">📝 Notes</h3>
          <ul className="list-disc pl-5 space-y-1 text-sm">
            {notes.map((n, i) => (<li key={i}>{n}</li>))}
          </ul>
        </div>
      )}
    </div>
  );
}
```

frontend/src/components/ChatWizard.jsx

```
import React, { useState, useMemo, useEffect } from 'react';
import { SignedIn, SignedOut, SignInButton } from '@clerk/clerk-react';
import { api } from '../lib/api';
import ItineraryView from './ItineraryView';

const LOCATION_GROUPS = {
  countries: ['Germany', 'France', 'United Kingdom', 'Belgium', 'Italy',
'Spain', 'Switzerland'],
  regions: ['Europe', 'Asia', 'South America', 'North America', 'Africa']
};

export default function ChatWizard() {
  const [step, setStep] = useState(1);
```

```
  const [destinations, setDestinations] = useState([]);
  const [suggestion, setSuggestion] = useState(null);
  const [days, setDays] = useState('');
  const [flexible, setFlexible] = useState(false);
  const [flexStart, setFlexStart] = useState(false);
  const [flexEnd, setFlexEnd] = useState(false);
  const [startDate, setStartDate] = useState('');
  const [endDate, setEndDate] = useState('');
  const [adults, setAdults] = useState(1);
  const [children, setChildren] = useState(0);
  const [tripType, setTripType] = useState('Nature Lover');
  const [budget, setBudget] = useState('Economical');
  const [stay, setStay] = useState('Hotels');
  const [flightPref, setFlightPref] = useState('Direct only');
  const [localTransport, setLocalTransport] = useState('Public Transport');
  const [needs, setNeeds] = useState({ visa: true, insurance: true,
translation: false });

  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [result, setResult] = useState(null);

  const computedEndDate = useMemo(() => {
    if (!startDate || !days) return '';
    const d = new Date(startDate);
    const n = parseInt(days, 10);
    if (Number.isNaN(n)) return '';
    d.setDate(d.getDate() + (n - 1));
    return d.toISOString().slice(0, 10);
  }, [startDate, days]);

  useEffect(() => {
    if (!endDate && computedEndDate) setEndDate(computedEndDate);
  }, [computedEndDate, endDate]);

  function toggleList(list, value) {
    return list.includes(value) ? list.filter(x => x !== value) : [...list,
value];
  }

  async function handleDestinationsSubmit() {
    try {
      setLoading(true); setError('');
      const data = await api.suggest(destinations);
      setSuggestion(data);
      const firstNum = String(data.recommended_days || '').match(/\d+/)?.[0];
      if (firstNum) setDays(firstNum);
      setStep(2);
```

```
    } catch (e) {
      setError(e.message);
    } finally { setLoading(false); }
  }

  async function handleGenerate() {
    try {
      setLoading(true); setError('');
      const payload = {
        destinations,
        duration_days: Number(days),
        duration_flexible: flexible,
        dates: { start: startDate, end: endDate, flex_start: flexStart,
flex_end: flexEnd },
        travelers: { adults, children_under5: children },
        trip_type: tripType,
        budget,
        accommodation: stay,
        flight_preference: flightPref,
        local_transport: localTransport,
        vegetarian_jain: true,
        needs,
      };
      const data = await api.itinerary(payload);
      setResult(data);
      setStep(99);
    } catch (e) {
      setError(e.message);
    } finally { setLoading(false); }
  }

  return (
    <div className="max-w-3xl mx-auto p-4 text-slate-100">
      <div className="mb-6 flex items-center justify-between">
        <h1 className="text-2xl font-semibold">🏛️ Voyager AI – Trip Planner</h1>
        <SignedOut><SignInButton mode="modal"><button className="px-4 py-2
rounded-xl bg-sky-600 hover:bg-sky-700">Sign in</button></SignInButton></
SignedOut>
      </div>

      {error && (<div className="mb-4 p-3 rounded-lg bg-red-900/40 border
border-red-800 text-sm">{error}</div>)}

      <SignedIn>
        {step === 1 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">1) Where do you want to
```

```
travel?</h2>
            <div className="text-sm text-slate-400">Choose <b>Specific
locations</b> (countries) and/or <b>Regions</b>. No free text.</div>
            <div className="grid md:grid-cols-2 gap-4">
              <div>
                <div className="text-slate-400 mb-2">Specific locations</div>
                <div className="flex flex-wrap gap-2">
                  {LOCATION_GROUPS.countries.map(c => (
                    <button key={c} onClick={() => setDestinations(prev =>
toggleList(prev, c))}
                      className={`px-3 py-1.5 rounded-full border $
{destinations.includes(c) ? 'bg-sky-600 border-sky-500' : 'bg-slate-950/60
border-slate-800'}`}>{c}</button>
                  ))}
                </div>
              </div>
              <div>
                <div className="text-slate-400 mb-2">Regions</div>
                <div className="flex flex-wrap gap-2">
                  {LOCATION_GROUPS.regions.map(r => (
                    <button key={r} onClick={() => setDestinations(prev =>
toggleList(prev, r))}
                      className={`px-3 py-1.5 rounded-full border $
{destinations.includes(r) ? 'bg-sky-600 border-sky-500' : 'bg-slate-950/60
border-slate-800'}`}>{r}</button>
                  ))}
                </div>
              </div>
            </div>
            <div className="flex gap-3">
              <button disabled={loading || destinations.length === 0}
onClick={handleDestinationsSubmit}
                className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-700
disabled:opacity-50">Get suggestions</button>
            </div>
            {loading && <div className="text-sm text-slate-400">Thinking…</div>}
            {suggestion && (
              <div className="mt-4 text-sm">
                <div className="text-slate-400">Gemini suggests:</div>
                <div>Days: <span className="font-
medium">{suggestion.recommended_days}</span></div>
                <div>Best Months: <span className="font-
medium">{suggestion.best_months}</span></div>
                <div>Budget: <span className="font-
medium">{suggestion.estimated_budget}</span></div>
              </div>
            )}
          </div>
```

```
        )}

        {step === 2 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">2) How many days?</h2>
            <input type="number" min="1" value={days} onChange={e =>
setDays(e.target.value)}
              className="px-3 py-2 rounded-lg bg-slate-950/60 border border-
slate-800" />
            <label className="flex items-center gap-2 text-sm">
              <input type="checkbox" checked={flexible} onChange={e =>
setFlexible(e.target.checked)} /> Flexible (+/- 2 days)
            </label>
            <div className="flex gap-3">
              <button onClick={() => setStep(1)}
className="px-4 py-2 rounded-xl bg-slate-800">Back</button>
              <button onClick={() => setStep(3)}
className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-700">Next</button>
            </div>
          </div>
        )}

        {step === 3 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">3) Dates</h2>
            <div className="grid md:grid-cols-2 gap-3">
              <div>
                <div className="text-sm text-slate-400 mb-1">Start date</div>
                <input type="date" value={startDate} onChange={e =>
setStartDate(e.target.value)}
                  className="w-full px-3 py-2 rounded-lg bg-slate-950/60 border
border-slate-800"/>
              </div>
              <div>
                <div className="text-sm text-slate-400 mb-1">End date (auto)</
div>
                <input type="date" value={endDate} onChange={e =>
setEndDate(e.target.value)}
                  className="w-full px-3 py-2 rounded-lg bg-slate-950/60 border
border-slate-800"/>
              </div>
            </div>
            {flexible && (
              <div className="grid md:grid-cols-3 gap-3 text-sm">
                <label className="flex items-center gap-2"><input
type="checkbox" checked={flexStart} onChange={e =>
```

```
setFlexStart(e.target.checked)} /> Start can shift</label>
                <label className="flex items-center gap-2"><input
type="checkbox" checked={flexEnd} onChange={e => setFlexEnd(e.target.checked)} /
> End can shift</label>
                <div className="text-slate-400">(Everything is flexible)</div>
              </div>
            )}
            {suggestion?.best_months && (
              <div className="text-sm text-slate-400">Preferred time to visit:
{suggestion.best_months}</div>
            )}
            <div className="flex gap-3">
              <button onClick={() => setStep(2)}
className="px-4 py-2 rounded-xl bg-slate-800">Back</button>
              <button onClick={() => setStep(4)}
className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-700">Next</button>
            </div>
          </div>
        )}

        {step === 4 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">4) Travelers</h2>
            <div className="flex items-center gap-4">
              <div className="flex items-center gap-2">
                <span>Adults</span>
                <div className="flex items-center gap-2">
                  <button onClick={() => setAdults(Math.max(1, adults - 1))}
className="px-2 py-1 rounded bg-slate-800">-</button>
                  <span className="w-6 text-center">{adults}</span>
                  <button onClick={() => setAdults(adults + 1)} className="px-2
py-1 rounded bg-slate-800">+</button>
                </div>
              </div>
              <div className="flex items-center gap-2">
                <span>Children (&lt;5)</span>
                <div className="flex items-center gap-2">
                  <button onClick={() => setChildren(Math.max(0, children -
1))} className="px-2 py-1 rounded bg-slate-800">-</button>
                  <span className="w-6 text-center">{children}</span>
                  <button onClick={() => setChildren(children + 1)}
className="px-2 py-1 rounded bg-slate-800">+</button>
                </div>
              </div>
            </div>
            <div className="flex gap-3">
              <button onClick={() => setStep(3)}
```

```
className="px-4 py-2 rounded-xl bg-slate-800">Back</button>
              <button onClick={() => setStep(5)}
className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-700">Next</button>
            </div>
          </div>
        )}

        {step === 5 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">5) Trip Type</h2>
            <div className="flex flex-wrap gap-2">

{['Adventure','Relaxing','Romantic','Pilgrimage','Family','Business','Nature
Lover'].map(t => (
                <button key={t} onClick={() => setTripType(t)}
                  className={`px-3 py-1.5 rounded-full border ${tripType ===
t ? 'bg-sky-600 border-sky-500' : 'bg-slate-950/60 border-slate-800'}`}>{t}</
button>
              ))}
            </div>
            <div className="flex gap-3">
              <button onClick={() => setStep(4)}
className="px-4 py-2 rounded-xl bg-slate-800">Back</button>
              <button onClick={() => setStep(6)}
className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-700">Next</button>
            </div>
          </div>
        )}

        {step === 6 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">6) Budget</h2>
            <div className="flex flex-wrap gap-2">
              {['Saver','Economical','Premium','No Limit'].map(b => (
                <button key={b} onClick={() => setBudget(b)}
                  className={`px-3 py-1.5 rounded-full border ${budget === b ?
'bg-sky-600 border-sky-500' : 'bg-slate-950/60 border-slate-800'}`}>{b}</button>
              ))}
            </div>
            <div className="flex gap-3">
              <button onClick={() => setStep(5)}
className="px-4 py-2 rounded-xl bg-slate-800">Back</button>
              <button onClick={() => setStep(7)}
className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-700">Next</button>
            </div>
          </div>
```

```
        )}

        {step === 7 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">7) Accommodation</h2>
            <div className="flex flex-wrap gap-2">
              {['Hotels','Airbnb','Resorts','Hostels'].map(s => (
                <button key={s} onClick={() => setStay(s)}
                  className={`px-3 py-1.5 rounded-full border ${stay === s ?
'bg-sky-600 border-sky-500' : 'bg-slate-950/60 border-slate-800'}`}>{s}</button>
              ))}
            </div>
            <div className="flex gap-3">
              <button onClick={() => setStep(6)}
className="px-4 py-2 rounded-xl bg-slate-800">Back</button>
              <button onClick={() => setStep(8)}
className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-700">Next</button>
            </div>
          </div>
        )}

        {step === 8 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">8) Flights</h2>
            <div className="flex flex-wrap gap-2">
              {['Direct only','Cheapest','Flexible'].map(s => (
                <button key={s} onClick={() => setFlightPref(s)}
                  className={`px-3 py-1.5 rounded-full border ${flightPref ===
s ? 'bg-sky-600 border-sky-500' : 'bg-slate-950/60 border-slate-800'}`}>{s}</
button>
              ))}
            </div>
            <div className="flex gap-3">
              <button onClick={() => setStep(7)}
className="px-4 py-2 rounded-xl bg-slate-800">Back</button>
              <button onClick={() => setStep(9)}
className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-700">Next</button>
            </div>
          </div>
        )}

        {step === 9 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">9) Local Transport</h2>
            <div className="flex flex-wrap gap-2">
```

```
              {['Public Transport','Car Rental','Private Taxi'].map(s => (
                <button key={s} onClick={() => setLocalTransport(s)}
                  className={`px-3 py-1.5 rounded-full border ${localTransport
=== s ? 'bg-sky-600 border-sky-500' : 'bg-slate-950/60 border-slate-800'}`}>{s}
</button>
              ))}
            </div>
            <div className="flex gap-3">
              <button onClick={() => setStep(8)}
className="px-4 py-2 rounded-xl bg-slate-800">Back</button>
              <button onClick={() => setStep(10)} className="px-4 py-2 rounded-
xl bg-sky-600 hover:bg-sky-700">Next</button>
            </div>
          </div>
        )}

        {step === 10 && (
          <div className="space-y-4 rounded-2xl p-6 bg-slate-900/60 border
border-slate-800">
            <h2 className="text-lg font-semibold">10) Special Considerations</
h2>
            <div className="grid md:grid-cols-3 gap-3 text-sm">
              <label className="flex items-center gap-2"><input type="checkbox"
checked={needs.visa} onChange={e => setNeeds({ ...needs, visa:
e.target.checked })}/> Visa guidance</label>
              <label className="flex items-center gap-2"><input type="checkbox"
checked={needs.insurance} onChange={e => setNeeds({ ...needs, insurance:
e.target.checked })}/> Travel insurance</label>
              <label className="flex items-center gap-2"><input type="checkbox"
checked={needs.translation} onChange={e => setNeeds({ ...needs, translation:
e.target.checked })}/> Language support</label>
            </div>
            <div className="flex gap-3">
              <button onClick={() => setStep(9)}
className="px-4 py-2 rounded-xl bg-slate-800">Back</button>
              <button disabled={loading} onClick={handleGenerate}
className="px-4 py-2 rounded-xl bg-emerald-600 hover:bg-emerald-700
disabled:opacity-50">Generate Itinerary</button>
            </div>
            {loading && <div className="text-sm text-slate-400">Generating…</
div>}
          </div>
        )}

        {step === 99 && (
          <div className="space-y-4">
            <div className="rounded-2xl p-6 bg-slate-900/60 border border-
slate-800">
```

```
              <h2 className="text-lg font-semibold mb-2">Your Itinerary</h2>
              <ItineraryView data={result} />
            </div>
            <button onClick={() => { setResult(null); setStep(1); }}
className="px-4 py-2 rounded-xl bg-slate-800">Plan another trip</button>
          </div>
        )}
      </SignedIn>

      <SignedOut>
        <div className="rounded-2xl p-6 bg-slate-900/60 border border-
slate-800">Please sign in to plan a trip.</div>
      </SignedOut>
    </div>
  );
}
```

---

3) Environment setup

Backend (`server/`):

```
cd server
npm install
cp .env.example .env
# fill in keys
npm run dev
```

Frontend (`frontend/`):

```
cd frontend
npm install
# .env for Vite
printf "VITE_CLERK_PUBLISHABLE_KEY=pk_test_xxx\nVITE_API_BASE=http://localhost:
5000\n" > .env.local
npm run dev
```

Open http://localhost:5173, sign in with Clerk, walk through the wizard.

---

4) Smoke tests

Backend direct (replace TOKEN with a real Clerk session JWT):

```
curl -X POST http://localhost:5000/api/suggest
 -H "Authorization: Bearer TOKEN" -H "Content-Type: application/json"
 -d '{"destinations":["Germany","France"]}'

curl -X POST http://localhost:5000/api/itinerary
 -H "Authorization: Bearer TOKEN" -H "Content-Type: application/json"
 -d '{"destinations":["Germany","France"],"duration_days":
10,"duration_flexible":true,"dates":
{"start":"2025-09-01","end":"2025-09-11","flex_start":true,"flex_end":true},"travelers":
{"adults":2,"children_under5":
1},"trip_type":"Family","budget":"Economical","accommodation":"Hotels","flight_preference":"Direc
only","local_transport":"Public Transport","vegetarian_jain":true,"needs":
{"visa":true,"insurance":true,"translation":false}}'
```

> When calling APIs from the browser app, the Clerk SDK sends the cookie/session for you; you don't need to manually pass the token.

---

5) How the flow works (quick recap)

1) **Destinations** → `/api/suggest` (Gemini JSON: days / months / budget)

2) **Days** (+/- 2 days flexible)

3) **Dates** (end auto = start + days – 1; optional flexibility flags; show suggested best months)

4) **Travelers** (adults / children under 5)

5) **Trip type**, 6) **Budget**, 7) **Stay**, 8) **Flights**, 9) **Local transport**, 10) **Special needs**

11) **Generate** → `/api/itinerary` (Gemini full structured JSON) → **render** in `ItineraryView`.

Diet: Always send `vegetarian_jain: true`.

---

6) Notes & troubleshooting

- **Clerk SDK**: `@clerk/clerk-sdk-node` is deprecated (EOL Jan 10, 2025). This pack uses `@clerk/express` + `@clerk/backend` and a strict `mustBeAuthed` middleware that returns **401** JSON for API routes (no redirects). See Clerk docs.
- **CORS**: Set `CLIENT_ORIGIN` (comma-separated if multiple). Frontend must request with `credentials: 'include'` (already handled in `api.js`).

- **Gemini JSON**: The helper strips `` ` ``` fences and tries to parse. If you want stricter outputs, consider Gemini **JSON mode** or add retries with a looser regex + zod` validation.
- **Dates**: End date is auto-computed but still editable by the user.
- **Regions vs Countries**: We pass them as selected; the prompt lets Gemini infer routes.
- **Environment keys**: `VITE_` prefix is required for Vite to expose the publishable key to the client.

---

7) Optional upgrades

- **Streaming** (SSE) for `/api/itinerary` to show progressive rendering.
- **Persist trips** to a DB keyed by Clerk user ID.
- **More destinations** with search and tags.
- **Price lookups** with flight/hotel APIs (Skyscanner, Amadeus, etc.)

Done ✅

Drop these files into your repo and run both servers. You'll have a **guided, Clerk-protected, Gemini-powered itinerary generator** with a polished UI.